Day 7 Report

DEVOPS-007: Dockerfile & Custom Docker Image | Task 11: Documentation | Time: 45 Minutes

1. What is Dockerfile?

A Dockerfile is a plain text file containing a set of step-by-step instructions that Docker reads to automatically build a custom image. Each instruction (FROM, COPY, RUN, etc.) defines part of how the image is assembled and how a container from it should behave. It makes image creation repeatable, version-controlled, and shareable, instead of manually configuring containers by hand.

2. Dockerfile Instructions

Instruction	Purpose
`FROM`	Sets the base image the new image is built on top of
`WORKDIR`	Sets the working directory for later instructions inside the container
`COPY`	Copies files/folders from the host into the image
`RUN`	Executes a command at build time and bakes the result into a new layer
`CMD`	Sets the default command run when a container starts (overridable)
`ENTRYPOINT`	Sets the fixed main command a container always runs (hard to override)
`EXPOSE`	Documents which port(s) the container listens on

3. Docker Build Process

Running docker build -t <name>:<tag> . tells Docker to read the Dockerfile in the current directory (the build context) and execute each instruction top to bottom, creating a new filesystem layer per instruction. Layers are cached, so unchanged steps are reused on future builds, making rebuilds faster. Once every instruction completes, Docker produces a final image tagged with the name/tag given, ready to run as a container.

```
docker build -t acme-website:v1 .
```

4. Docker Image Versions

Built multiple tagged versions of the same image to track changes over time instead of overwriting a single image:

```
docker build -t devops-website:v1 .
docker build -t devops-website:v2 .
docker build -t devops-website:latest .
```

latest is just a conventional tag, not automatically the newest build — it has to be built/pushed deliberately. Meaningful version tags (v1, v2, or commit-hash based tags) make deployments traceable and reversible, since a specific known-good version can always be redeployed if something breaks.

5. .dockerignore

```
.git
*.md
node_modules
.DS_Store
```

Works like .gitignore but for Docker builds — it excludes specified files/folders from the build context sent to the Docker daemon. This keeps images smaller, builds faster, and prevents sensitive or unnecessary files from accidentally being copied in.

6. Port Mapping

```
docker run -d --name acme-web -p 8080:80 acme-website:v1
```

The -p flag maps a host port to a container port in the format host:container. Here, requests to localhost:8080 on the host machine are forwarded to port 80 inside the container, where Nginx is listening. Without this mapping, the container's internal port isn't reachable from outside the container.

7. Nginx Container

Used the nginx:alpine base image to containerize a small static company website. Nginx serves files placed in /usr/share/nginx/html by default, so the Dockerfile copies the site's index.html and style.css into that directory and exposes port 80.

```
FROM nginx:alpine
WORKDIR /usr/share/nginx/html
COPY index.html .
COPY style.css .
EXPOSE 80
```

8. Docker Hub

Docker Hub is a public (or private) registry for storing and sharing Docker images. To push an image, it must first be logged in to and tagged in the format <username>/<repository>:<tag>:

```
docker login
docker tag acme-website:v2 mydockerhubuser/acme-website:v2
docker push mydockerhubuser/acme-website:v2
```

Once pushed, the image can be pulled and run on any machine with Docker installed, confirming the image is portable.

9. Troubleshooting

Deliberately introduced errors into the Dockerfile to practice reading build failures and fixing them methodically: wrong base image tag, wrong COPY path, wrong filename, wrong command/package name, and wrong port. Each was diagnosed using the build error output together with docker images, docker ps -a, docker logs, and docker inspect, then fixed and re-verified with a rebuild.

10. Problems Faced

Problem	Error Message
Wrong base image tag (22.05)	manifest unknown / no match for platform in manifest

Wrong COPY path (srcfiles/)	failed to calculate checksum of ref: not found
Wrong filename (index.htm)	failed to calculate checksum of ref: not found
Wrong package name (ngnix)	E: Unable to locate package ngnix
Wrong EXPOSE port	curl: Connection refused

11. Solutions

Problem	Fix
Wrong base image tag	Changed to a valid tag: ubuntu:22.04
Wrong COPY path	Corrected path to match the real folder name (src/)
Wrong filename	Corrected extension to .html
Wrong package name	Fixed the typo: ngnix → nginx
Wrong EXPOSE port	Set EXPOSE 80 to match Nginx's real listening port

13. What I Learned

Today reinforced how a Dockerfile turns manual container setup into a repeatable, version-controlled process. Key takeaways: FROM/WORKDIR/COPY/RUN/CMD/EXPOSE each play a distinct role in building an image; RUN executes at build time while CMD only defines the runtime default (and ENTRYPOINT is the harder-to-override version of that); meaningful version tags matter far more than relying on latest because they make deployments traceable and rollback possible; and most build errors (bad base image, wrong paths, typos, wrong ports) are diagnosable by carefully reading the error message and checking it against docker images / docker ps -a / docker inspect before jumping to conclusions.